

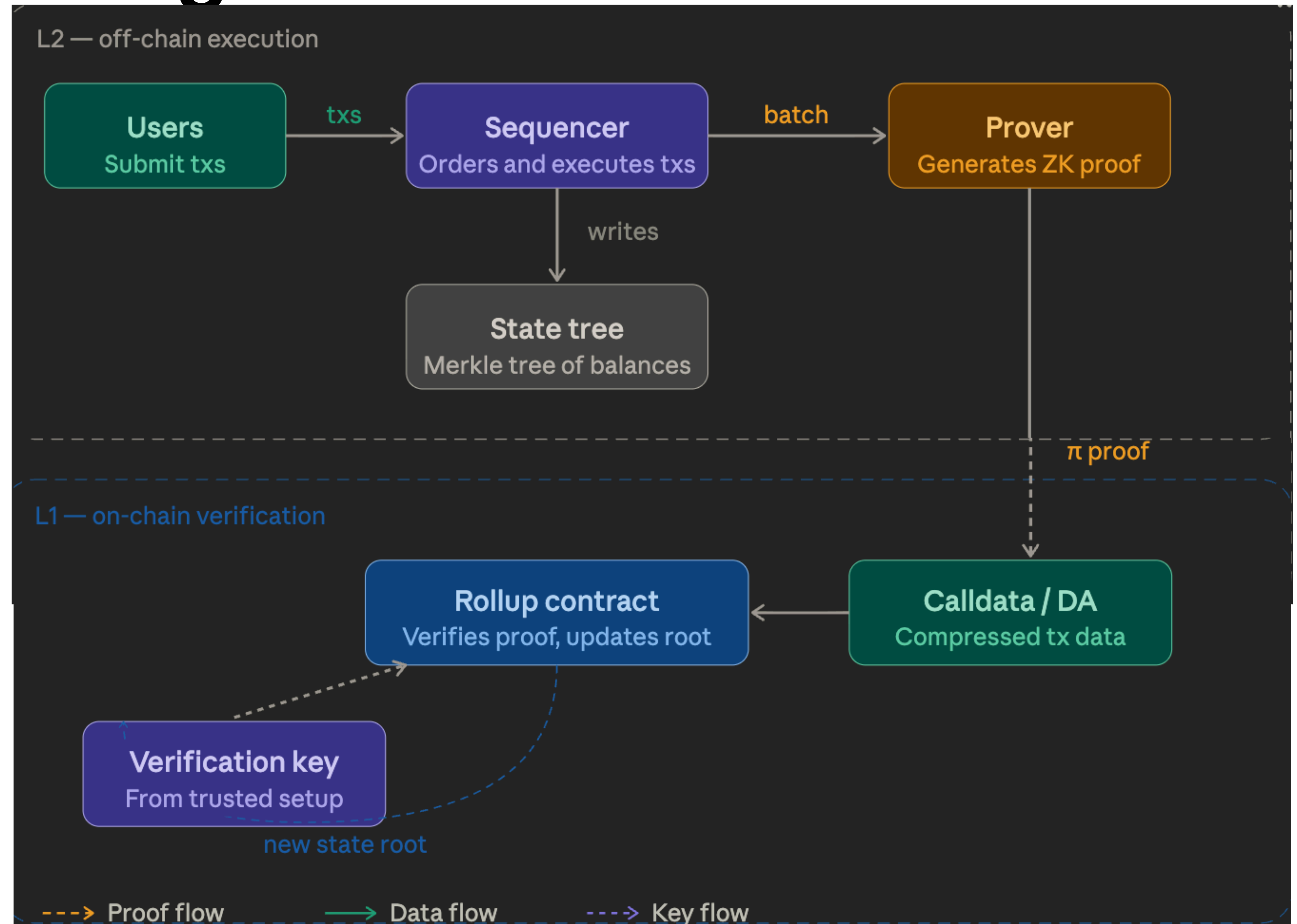
Introduction to Blockchain and Smart Contracts

Lecture 15-16: ZK Rollups (Intro to ZKP)

Subodh Sharma | IIT Delhi | April 02, 06 2026



ZK- Rollups: A High Level View



The Mathematics behind ZKPs

- **What is a ZK Proof?**
 - A protocol between a **prover P** and **verifier V**
 - **P** wants to convince **V** about the validity of a statement without revealing why is it true (the **witness**)
 - Formally: for a relation $R(x, w)$ where x is a public input and w is private
 - **Completeness:** If $R(x, w) = \text{True}$, an honest prover can always convince an honest verifier
 - **Soundness:** if $R(x, w) = \text{False}$ for all w , no cheating prover can convince **V**
 - **Zero-knowledge:** **V** learns nothing about w beyond the fact that $R(x, w) = \text{True}$

The Mathematics behind ZKPs

- **Example of a ZK Proof:**

- Anita wants to show that she knows the private key sk for an Ethereum address without revealing sk
- $R(addr, sk) := (keccak(G.sk) = addr)$
- IN ZK rollups: the witness w is the entire batch of txns + Merkle proof for every account touched. The statement x is $(r_{old}, r_{new}, batch_{hash})$

The Mathematics behind ZKPs

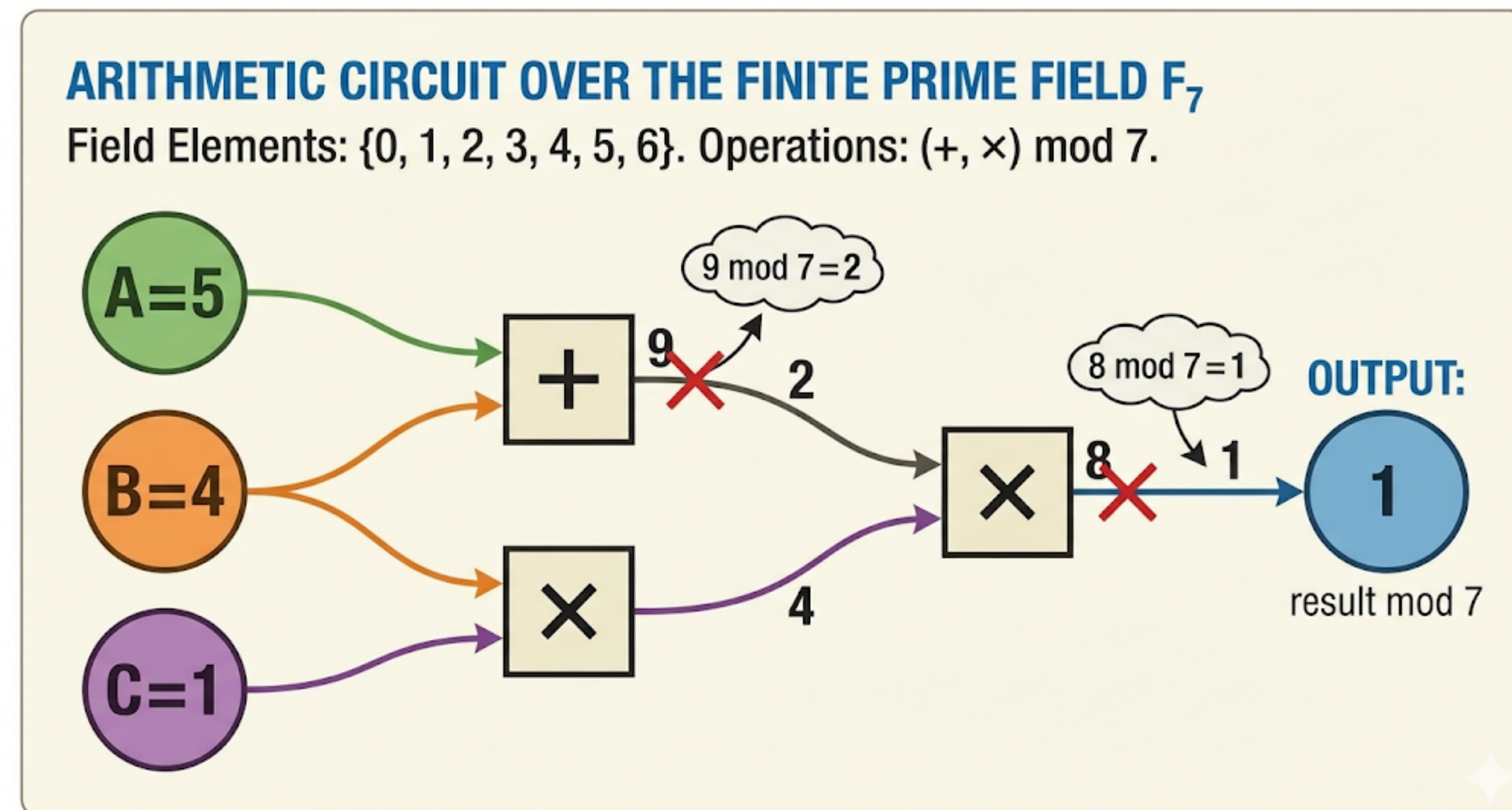
Finite Fields

- The basic algebraic universe used in ZKPs: Finite Fields over a prime $\mathbb{F}_p$
 - Finite fields allow addition, multiplication, subtraction and division among the elements
 - Closure, additive and multiplicative identities, additive and multiplication inverses
- Why choose fields?
 - Circuits, constraints and polynomials become algebraic equations over field elements

The Mathematics behind ZKPs

Circuits —> First step from program execution to algebraic constraints

- The prover typically expresses the computation as an arithmetic circuit over $\mathbb{F}_p$
- Usually the ckts are large:
 - Polygon zkEVM ~50 million constraints for an EVM step
- Eg: to compute $z = (a + b) \cdot c$
 - $t = a + b$
 - Constraints:
 - $t - a - b = 0$
 - $z - t \cdot c = 0$



The Mathematics behind ZKPs

Circuits —> in the context of ZK Roll ups

- Rollup conditions/constraints are boolean
 - Signature valid or invalid?
 - Nonce matches or not?
 - Balance sufficient or not?
- Boolean encoding
 - $b \cdot (b - 1) = 0$ (to encode boolean bit — roots are 0 or 1)
 - $x' = u \cdot x_{new} + (1 - u) \cdot x_{old}$ (encoding a conditional update)

The Mathematics behind ZKPs

R1CS - Rank-1 Constraint System

- 1st step: Program to a Ckt
- 2nd step: Ckt to an IR (R1CS) as a system of vectors (or constraints) in the form
 - $(A \cdot w) \times (B \cdot w) = C \cdot w$
- **Why R1CS?**
 - Compatibility with proof systems like Groth16 and Bulletproofs
 - Checking polynomials is faster than checking individual gates in a ckt
 - **Efficiency in verification (To be discussed later)**

The Mathematics behind ZKPs

R1CS - Rank-1 Constraint System

- Example: Prover knows x for which this is true: $x^3 + x + 5 = 35$
- Variables: $v = [1, x, s_1, s_2, out]$
 - Where the three constraints are:
 - $s_1 = x \cdot x,$
 - $s_2 = x \cdot s_1,$
 - $s_2 + x + 5 = out$

The Mathematics behind ZKPs

R1CS - Rank-1 Constraint System

- For constraint: $s_1 = x \cdot x$,
 - R1CS vectors: $\langle 0, 1, 0, 0, 0 \rangle$, $\langle 0, 1, 0, 0, 0 \rangle$, $\langle 0, 0, 1, 0, 0 \rangle$
 - $[\langle 0, 1, 0, 0, 0 \rangle , v] \cdot [\langle 0, 1, 0, 0, 0 \rangle , v] = [\langle 0, 0, 1, 0, 0 \rangle , v]$
 - Similarly for others

The Mathematics behind ZKPs

From R1CS to Polynomials

- R1CS must be encoded into polynomials so that prover can commit a solution without revealing it
 - But why polynomials?
 - **Data Compression:** Instead of sending 1,000,000 values, I can send you a "commitment" to one polynomial.
- A vector $(v_0, v_1, \dots, v_{n-1})$ can be encoded as the unique degree $n - 1$ polynomial, say $p(X)$ s.t. it evaluates to v_i at the i -th root of unity (this is interpolation)
- Eg: $v = (2, 5, 1, 6)$, then $p(X) = a_0 \cdot x^3 + a_1 \cdot x^2 + a_2 \cdot x + a_3$; Solve for the coefficients s.t. $p(\omega^0) = 2 \dots$

The Mathematics behind ZKPs

DeMillo–Lipton–Schwartz–Zippel lemma

- A non-zero polynomial of degree d can have at most d roots
- Lemma states: two distinct d -degree polynomials can intersect **at most** d times.
 - Let $p(X)$: The "True" polynomial representing the correct solution
 - Let $q(X)$: The "Prover's" polynomial
- The sampling: Imagine universal of possible X is very large; if prover is lying then only a small d points exists where the prover can lie successfully;
 - If the verifier picks up one random point from the field and the math holds up then the probability of prover being a liar is negligible.

The Mathematics behind ZKPs

Polynomial Commitments

- A verifier cannot ask the prover to send giant polynomials explicitly. So proof systems use **polynomial commitments**:
 - short objects that commit to a polynomial
 - Typical solution in ZK rollups: **Kate–Zaverucha–Goldberg** commitments (built on **elliptic curve pairings**)
 - Takes as input security parameter and the maximum degree of the polynomial
 - Outputs a Prover key and the Verifier key
 - Prover uses Prover Key to cryptographically commit the secret polynomial and sends to the Verifier

The Mathematics behind ZKPs

Polynomial Commitments

- Typical solution in ZK rollups: **Kate–Zaverucha–Goldberg** commitments (built on **elliptic curve pairings**)
 - Takes as input security parameter and the maximum degree of the polynomial
 - Outputs a Prover key and the Verifier key
 - Prover uses Prover Key to cryptographically commit the secret polynomial f and sends to the Verifier
 - The verifier sends a challenge: a random point z to the prover
 - The prover calculates $f(z) = y$; the prover has to convince the verifier that y is indeed the correct o/p

The Mathematics behind ZKPs

Polynomial Commitments

- Typical solution in ZK rollups: **Kate–Zaverucha–Goldberg** commitments (built on **elliptic curve pairings**)
 - The prover calculates $f(z) = y$; the prover has to convince the verifier that y is indeed the correct o/p
 - The prover computes a proof of evaluation π in the following way:
 - If $f(z) = y$ then the polynomial $f(x) - y$ must have a root at $x = z$
 - Compute a quotient polynomial: $q(x) = (f(x) - y)/(x - z)$
 - Commit $q(x)$
 - The verifier verifies π

The Mathematics behind ZKPs

Polynomial Commitments

- The verifier obtains (y, π) and verifies using its Verifier key
 - Checks if the following **equality holds over committed data**: $f(x) - y = q(x) \cdot (x - z)$
 - But how does verifier accomplish the above?
 - Through **Bilinear maps: Elliptic curve pairings!**
 - If the math checks out, the Verifier is mathematically certain that $f(z) = y$ without ever seeing the coefficients of $f(x)$

The Mathematics behind ZKPs

Bilinear Pairings

- The pairing function: $G_1 \times G_2 \rightarrow G_T$ that satisfies the **bilinearity property**
 - **Bilinearity Property:** $e(P^a, Q^b) = e(P, Q)^{ab}$
- Assume the verifier has received the following:
 - C : The commitment of $f(x)$ at some secret s
 - π : The commitment of the quotient polynomial $q(x)$
 - y, z : Scalars (the evaluation and the evaluation point)
- The verifier needs to check $f(x) - y = q(x) \cdot (x - z)$

The Mathematics behind ZKPs

Bilinear Pairings

$$f(x) - y = q(x) \cdot (x - z)$$

- The verifier computes the difference over commitment: $C - [y]$
 - $[.]$ — means a scalar value mapped onto an elliptic curve
 - $C - [y]$ represents $[f(s) - y]$ in G_1
- The verifier then uses the proof π (which is $[q(s)]$). The public parameters provide $[s - z]$ in G_2 .
- Applying the bilinear property we have two equations (one for the left side of the equation and other for the right)
 - $e([f(s) - y], G_2) = e(G_1, G_2)^{f(s)-y}$
 - $e([q(s)], [s - z]) = e(G_1, G_2)^{q(s) \cdot (s-z)}$
 - If the above two expressions are equal, then verifier has confirmed the identity securely and privately!

The Mathematics behind ZKPs

Going from Interactive to Non-Interactive - Groth16

- What we saw was an interactive proof — prover commits, verifier sends a random challenge, and prover responds
- How to make the entire protocol **non-interactive**?
 - **Fiat-Shamir Transformation:** Key insight is to replace the verifier challenge with a hash function $H(x, s, \dots)$
 - Let us take the earlier example, prove that you know the rational roots of the equation $x^3 + x + 5 = 35$

Groth16

$$x^3 + x + 5 = 35$$

- Gates:
 - $S_1 = x \cdot x$
 - $S_2 = x \cdot S_1$
 - $y = S_2 + x + 5$
- R1CS:
 - Each gate in the constraint form: $(A \cdot s) \times (B \cdot s) = (C \cdot s)$
 - Where s is the witness vector $s = [1, x, sym1, sym2, out]$ and A, B, C are selectors
 - For gate 1: $A = [0, 1, 0, 0, 0], B = [0, 1, 0, 0, 0], C = [0, 0, 1, 0, 0]$

Groth16

$$x^3 + x + 5 = 35$$

- R1CS to QAP (Quadratic Arithmetic Program):
 -